

**IES POLITÉCNICO HERMENEGILDO LANZ
CFGS DESARROLLO DE APLICACIONES WEB**

Saal

PLATAFORMA DE COMPRA Y RESERVA DE VIAJES EN AUTOBÚS

**DOCUMENTACIÓN Y ANÁLISIS DEL PROYECTO DE
FIN DE GRADO**

**Autoría: Martín Marín Vilches
03/05/2026**

ÍNDICE

1. Análisis del problema	2
1.1. Introducción	2
1.2. Objetivos	3
1.3. Funciones y rendimientos deseados	3
1.4. Planteamiento y evaluación de diversas soluciones	3
1.5. Justificación de la solución elegida	3
1.6. Modelado de la solución	3
1.6.1. Recursos humanos	3
1.6.2. Recursos hardware	4
1.6.3. Recursos software	4
1.7. Planificación temporal	4
2. Diseño e implementación del proyecto	4
3. Fase de pruebas	4
4. Documentación de la aplicación	5
4.1. Introducción a la aplicación (Getting Started)	5
4.2. Manual de Instalación	5
4.3. Manual de usuario	5
4.4. Manual de administración	5
5. Conclusiones finales	5
6. Bibliografía	5

1. Análisis del problema

1.1. Introducción

Saal es un sistema integral de reserva y gestión de billetes de autobús diseñado para optimizar los servicios de transporte. La plataforma permite a los operadores de autobuses administrar rutas, estaciones y horarios, mientras que ofrece a los clientes una interfaz intuitiva para la búsqueda y compra de billetes.

1.2. Objetivos

El objetivo central es desarrollar un ecosistema diseñado para agilizar la gestión de movilidad y la comercialización de servicios de transporte. La plataforma busca integrar la administración inteligente de flotas y el procesamiento de ventas en tiempo real, eliminando las brechas operativas entre la logística y el punto de venta.

Saal también tiene como objetivo servir de herramienta en diversos ámbitos de la industria, a través de una arquitectura versátil, incluyendo:

- Empresas de transporte regular, optimizando sus rutas y frecuencias.
- Servicios de transporte corporativo y privado, garantizando puntualidad y trazabilidad.

Como objetivo final, Saal se proyecta como ejemplo de digitalización que permite a las organizaciones alcanzar una visibilidad total de su operación, mejorando la experiencia del usuario y maximizando la rentabilidad mediante la automatización y la sincronización de datos de extremo a extremo.

1.3. Funciones y rendimientos deseados

Para lograr un control total sobre la operación, Saal despliega sus servicios a través de cinco pilares que conectan la logística de campo con la estrategia de ventas:

- **Planificación Dinámica de Trayectos:** El sistema no se limita a fijar un punto A y un punto B. Permite estructurar redes complejas de transporte con múltiples escalas, donde cada estación intermedia cuenta con sus propios tiempos. La configuración de horarios es flexible, permitiendo ajustar frecuencias según el día de la semana, temporadas altas o necesidades específicas de cada ruta, asegurando que la oferta siempre coincida con la capacidad real de la flota.
- **Transaccionalidad Directa y Sincronizada:** la venta y reserva de billetes ocurre sin intermediarios que retrasen la información. Cuando un usuario reserva un asiento, el inventario se actualiza de inmediato para toda la red de ventas, ya sea en taquilla física o canales digitales. Esto elimina por completo el riesgo de duplicidad de asientos y permite una gestión de ingresos mucho más ágil, procesando pagos y confirmaciones al instante.
- **Centro de Control Administrativo:** es el núcleo desde donde se define el ritmo del negocio. Más que un simple panel de ajustes, es la herramienta que permite a los gerentes configurar calendarios laborales, asignar roles al personal y supervisar que todo el ecosistema funcione

bajo los parámetros establecidos. Es aquí donde se traduce la estrategia comercial en reglas operativas concretas para el sistema.

- **Red Logística de Estaciones:** Saal permite organizar y supervisar cada terminal o punto de parada de forma independiente pero integrada. Se pueden gestionar las particularidades de cada estación en distintas ciudades, facilitando la logística de despachos y asegurando que la comunicación entre las cabeceras de ruta y las paradas intermedias sea fluida y sin errores de coordinación.
- **Control de Activos y Diagnóstico de Ventas:** la plataforma cierra el círculo operativo vinculando el rendimiento comercial con la salud técnica de la flota. Permite actuar como un diario de mantenimiento que alerta sobre las revisiones necesarias de cada vehículo. El objetivo es evitar que una avería imprevista detenga la venta o que una mala planificación de horarios afecte la rentabilidad.

1.4. Planteamiento y evaluación de diversas soluciones

Para este proyecto se ha planteado una arquitectura moderna desacoplada que separa la lógica de negocio de la interfaz de usuario. Se evaluaron opciones monolíticas frente a arquitecturas basadas en API, optando por esta última para garantizar escalabilidad y mantenimiento.

1.5. Justificación de la solución elegida

Para el desarrollo del proyecto, se decidió por montar una API RESTful usando Symfony para el backend y conectarla con un frontend en React. La razón principal de esta mezcla es que se necesitaba una base sólida que no diera problemas al escalar. Symfony da una estructura robusta y profesional para manejar la lógica de negocio, mientras que React permite crear una interfaz mucho más fluida y, sobre todo, totalmente responsive, que hoy en día es clave para que los usuarios puedan usar la plataforma desde el móvil sin líos.

Además, al separar el sistema en una API, se asegura de que el software no sea un "bloque" cerrado. Esto facilita muchísimo que, en el futuro, se pueda conectar el sistema con otros servicios externos o incluso desarrollar una app móvil nativa consumiendo la misma base de datos de forma eficiente. Básicamente, se buscó una arquitectura moderna que sea fácil de mantener y que rinda bien bajo carga real.

1.6. Modelado de la solución

1.6.1. Recursos humanos

A la hora de diseñar el sistema, se valoró la variedad de perfiles y usuarios que puedan interactuar con el sistema, así que se preparó para ser funcional para todos. Por un lado están los administradores de transporte, que necesitan tener el control total de la logística y la flota; por otro, el personal de las agencias, que requiere una herramienta rápida para gestionar ventas y reservas sin complicaciones. Y, por supuesto, el usuario final, que lo único que busca es una interfaz sencilla e intuitiva para reservar su billete en un par de clics desde el móvil o el ordenador. La idea es que la plataforma sea lo suficientemente versátil para que cualquiera, sea experto o no, pueda navegar sin perderse.

1.6.2. Recursos hardware

Para el despliegue decidimos irnos por una arquitectura basada en contenedores, básicamente para no pelear con las configuraciones de cada entorno. Al usar Docker y Docker Compose, se asegura de que todo el stack (la base de datos, el backend y el frontend) funcione exactamente igual en el servidor que en nuestras máquinas. Lo único que se necesita es un servidor que sea compatible con Docker; con eso, levantar todos los servicios es súper rápido, se evitan conflictos de versiones y el mantenimiento del sistema se vuelve mucho más limpio y ordenado.

1.6.3. Recursos software

- **Backend:** PHP 8.2+ y framework Symfony.
- **Frontend:** React con diseño responsive y Node.js para desarrollo.
- **Base de datos:** MariaDB (puerto interno 3306) gestionado vía Docker.
- **Control de versiones:** Git.

1.7. Planificación temporal

El desarrollo del proyecto lo organizamos por fases para no dejar cabos sueltos. Empezando con el desarrollo de la API RESTful para tener toda la lógica del backend lista. continuando con la creación de las herramientas necesarias de administración del sistema para crear y gestionar datos de la API con agilidad. Se prosiguió con la implementación de la interfaz de usuario y por último se finalizó con la integración de las distintas partes del sistema, donde se aseguró que no hubiera agujeros de lógica o fallas en la navegación de la web..

2. Diseño e implementación del proyecto

A la hora de plantear cómo construir el sistema, se dividió el trabajo en varias etapas clave para asegurar que la arquitectura fuera sólida. La idea no era solo que el software funcionara, sino que fuera fácil de mantener y escalar.

1. La elección del Stack Tecnológico API RESTful con Symfony y un frontend en React responde a la necesidad de contar con un sistema robusto, que no solo sea rápido, sino que también tenga un diseño adaptable (*responsive*). Al separar el backend del frontend, asegurando que la aplicación sea eficiente y que, en un futuro, pueda integrarse con otros servicios de manera mucho más limpia, sin tener que rediseñar toda la estructura.

2. La arquitectura de despliegue se basa totalmente en contenedores. Esto obligó a configurar servidores compatibles con Docker y Docker Compose para ejecutar por separado los servicios de la base de datos, el backend y el frontend. Al usar contenedores, logramos que todo el stack sea mucho más fácil de levantar en cualquier entorno y que la gestión de dependencias sea mucho más ordenada.

3. En cuanto a perfiles de usuario, el diseño del sistema no es genérico; se pensó específicamente para tres tipos de roles que interactúan entre sí. Por un lado, están los administradores de transporte, que gestionan toda la logística pesada; el personal de las agencias, que necesita agilidad para vender; y finalmente el usuario final, para quien diseñamos una interfaz intuitiva que le permita hacer sus reservas sin complicaciones.

4. El desarrollo lo estructuramos en fases temporales para ir cumpliendo hitos de forma lógica:

- Empezamos con la configuración del entorno Docker para tener listo el laboratorio de desarrollo.
- Seguimos con el desarrollo de la API RESTful, estableciendo todos los *endpoints* necesarios.
- Después pasamos a la implementación de la interfaz de usuario en React.
- Y finalmente, nos centramos en la integración de los módulos de gestión de rutas, que es donde conectamos toda la lógica para que el sistema funcione como un ecosistema unificado.

5. Para que la documentación técnica sea útil y no un simple bloque de texto, la distribuimos en los directorios correspondientes. En el backend detallamos todo lo relacionado con Symfony y la API, mientras que en el frontend explicamos la lógica de visualización con React. Además, todos los detalles sobre el esquema de datos y cómo se estructuran las tablas se encuentran definidos en el archivo de inicialización de la base de datos, facilitando cualquier auditoría o mejora posterior del código.

3. Fase de pruebas

Como en cualquier desarrollo se sabía que iban a salir fallos, así que se plantea una batería de pruebas para intentar cazar todos los errores lógicos antes de dar el módulo por cerrado.

Para que esto no fuera un caos, usamos el sistema de *issues* de GitHub. Cada vez que se encontraba un bug o se nos ocurría una propuesta de mejora, abríamos un *issue* para tenerlo localizado. Esto permitió llevar un seguimiento continuo del proyecto, priorizar los fallos críticos y documentar cómo se iban solucionando. Al final, trabajar así ayudó a que el flujo de desarrollo fuera mucho más ordenado y a que no se olvidará ninguna mejora pendiente por el camino.

4. Documentación de la aplicación

4.1. Introducción a la aplicación (Getting Started)

Para poner en marcha Saal, lo primero es clonar el repositorio en tu equipo local. Hemos simplificado todo el proceso de instalación mediante Docker, por lo que no es necesario instalar Symfony o Node.js de forma manual; basta con ejecutar el comando para levantar los contenedores y el sistema se encargará de configurar el entorno por ti.

Una vez que los servicios estén arriba y corriendo, podrás acceder a la aplicación a través de las siguientes direcciones locales:

- **Interfaz de Usuario (Frontend):** <http://localhost:5173> – Aquí es donde verás la parte visual y podrás interactuar con el sistema como si fueras un usuario final o un agente.
- **Servicios y Datos (Backend API):** <http://localhost:8000> – Esta es la dirección de la API de Symfony, donde puedes consultar los *endpoints* o verificar que la lógica de datos está respondiendo correctamente.

4.2. Manual de Instalación

Lo que necesitas tener instalado previamente es **Git**, **Docker** y **Docker Compose**. Nos decantamos por este método porque te ahorra tener que configurar manualmente bases de datos o servidores locales.

El proceso para tener todo listo es bastante directo y se resume en estos tres pasos:

- **Clonado del proyecto:** Lo primero es bajar el código del repositorio a tu máquina local usando el comando `git clone`.
- **Lanzamiento del entorno:** Una vez dentro de la carpeta, solo hay que ejecutar el comando de Docker para levantar los servicios. Con esto, el backend, el frontend y la base de datos se configuran y se ponen en marcha simultáneamente en sus propios contenedores.
- **Preparación de los datos:** Por último, para que el sistema tenga dónde guardar la información, hay que ejecutar las **migraciones**. Este paso es fundamental porque es el que prepara el esquema de las tablas y deja la base de datos lista para empezar a trabajar.

4.3. Manual de usuario

Se diseñó una interfaz para que sea intuitiva según el rol que tengas en el sistema. La idea es que cada perfil encuentre lo que necesita de forma directa, sin dar vueltas por menús que no le corresponden:

- **Usuario Final:** Está pensado para que sea lo más sencillo posible. Cualquier persona puede entrar, buscar las **rutas destacadas** que ofrece el sistema y gestionar sus propias **reservas de billetes** en un par de pasos, recibiendo su confirmación al momento.
- **Agencias de Viajes:** Este perfil tiene herramientas un poco más potentes. El personal de la agencia puede gestionar **reservas grupales**, que suelen ser más complejas, y tiene permisos para tramitar **cancelaciones** o modificaciones de forma ágil cuando un cliente lo solicita.
- **Empresas y Transporte Corporativo:** Aquí el enfoque es más de control y logística. Las empresas pueden usar el sistema para **organizar los traslados de sus empleados** y, lo más importante, acceder a un **seguimiento de uso**. Así pueden ver exactamente cómo se están utilizando los servicios y llevar un control real de sus necesidades de transporte.

En resumen, el manual se divide en estos tres flujos de trabajo para que, dependiendo de quién use la aplicación, la experiencia sea lo más limpia y eficiente posible.

4.4. Manual de administración

El panel de administración del sistema está reservado para quienes gestionan la operativa de fondo. Aquí es donde se traduce la logística real al entorno digital a través de tres funciones clave:

- **Configuración de la Red de Estaciones:** No solo se trata de añadir puntos en un mapa; el administrador puede organizar las estaciones por ciudades, creando la estructura geográfica necesaria para que luego se puedan trazar las rutas.
- **Gestión del Calendario Operativo:** Desde este apartado se controla el calendario de trabajo de cada ruta. Se pueden definir qué días operan, en qué horarios y si hay cambios puntuales, asegurando que lo que el usuario ve en el frontend coincida con la realidad del servicio.

- **Supervisión y Control de Flota:** Esta herramienta permite monitorizar en todo momento la disponibilidad de los vehículos. El objetivo es que el administrador sepa qué unidades están activas y cuáles no, evitando asignar rutas a vehículos que estén en mantenimiento o fuera de servicio.

Básicamente, es una consola centralizada que permite que todo el sistema se mantenga actualizado y que la logística de transporte funcione sin errores de coordinación.

5. Conclusiones finales

Tras el desarrollo y las pruebas realizadas, podemos decir que **Saal** cumple con el objetivo principal que nos marcamos al principio: ofrecer una solución integral que realmente simplifica la gestión del transporte. Hemos logrado pasar de una operativa fragmentada a un ecosistema donde la administración de la flota y las ventas van de la mano en una misma plataforma, lo que hace que todo el proceso sea mucho más centralizado y, sobre todo, eficiente.

Aun así, siempre hay margen para seguir creciendo. De cara al futuro, se puede desarrollar diferentes líneas de expansión:

- **Analítica avanzada:** Potenciando la capacidad de análisis de los administradores para observar el rendimiento de la red, ayudar a predecir la demanda y optimizar aún más las rutas.
- **Itinerarios complejos:** Otra mejora clave sería permitir la personalización de viajes de varios días, algo que vendría genial para los operadores turísticos que gestionan rutas con pernoctación.

En definitiva, el proyecto sienta una base tecnológica sólida sobre la que se pueden seguir construyendo herramientas que se adapten a las necesidades reales del sector movilidad.